

**Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique**

Chapitre 3

Jeu d'instructions du PIC18F4550 et programmation Assembleur

1^{ère} année LSI

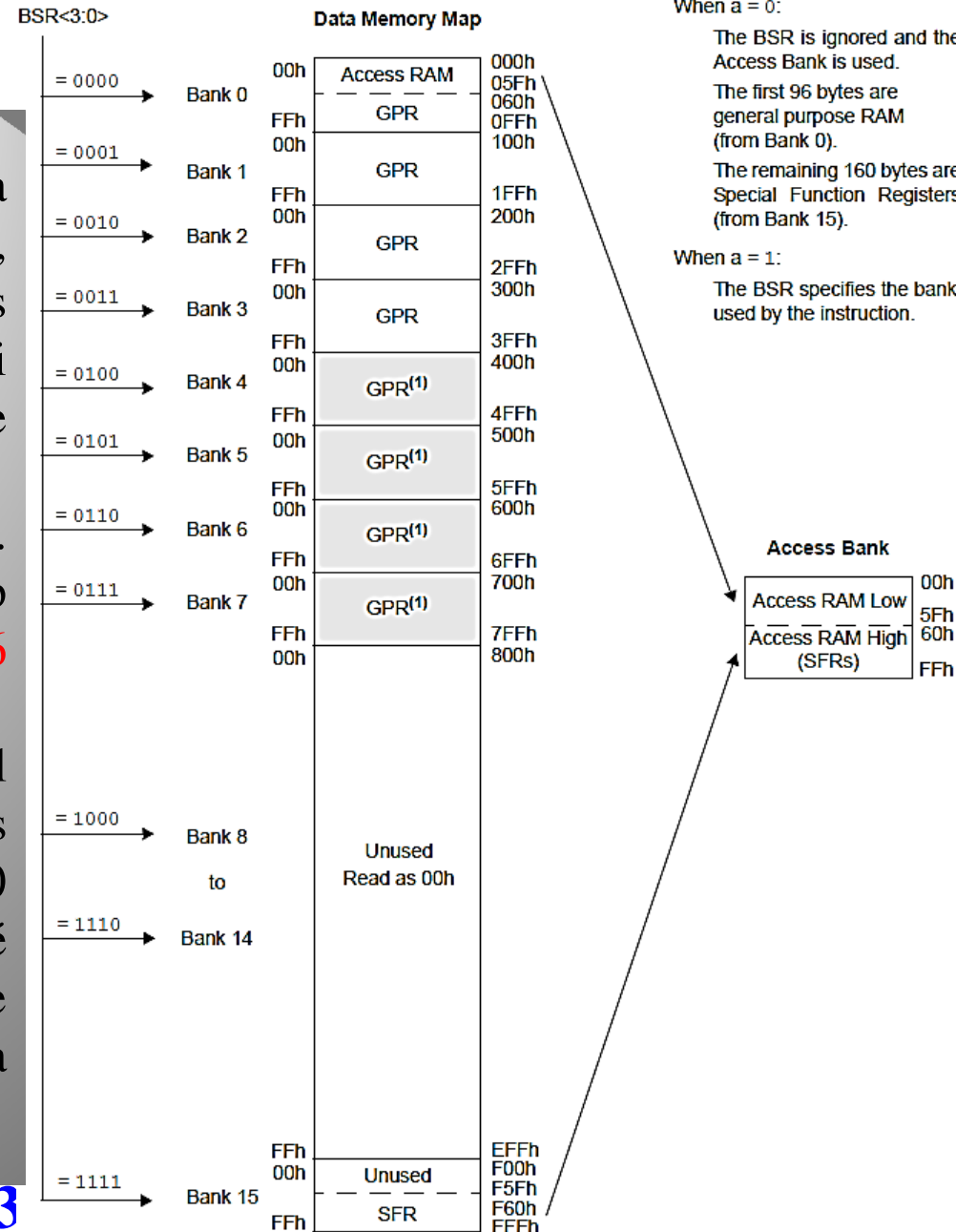
Plan du Cours

- Organisation de la mémoire de données et modes d'adressage
- Jeu d'instructions du PIC18F4550
- Les instructions « orientées Registres »
- Les instructions « orientées Bits »
- Les instructions de saut et appel de procédures
- Les instructions opérant sur une constante

Organisation de la mémoire de données et modes d'adressage

datasheet, page : 66

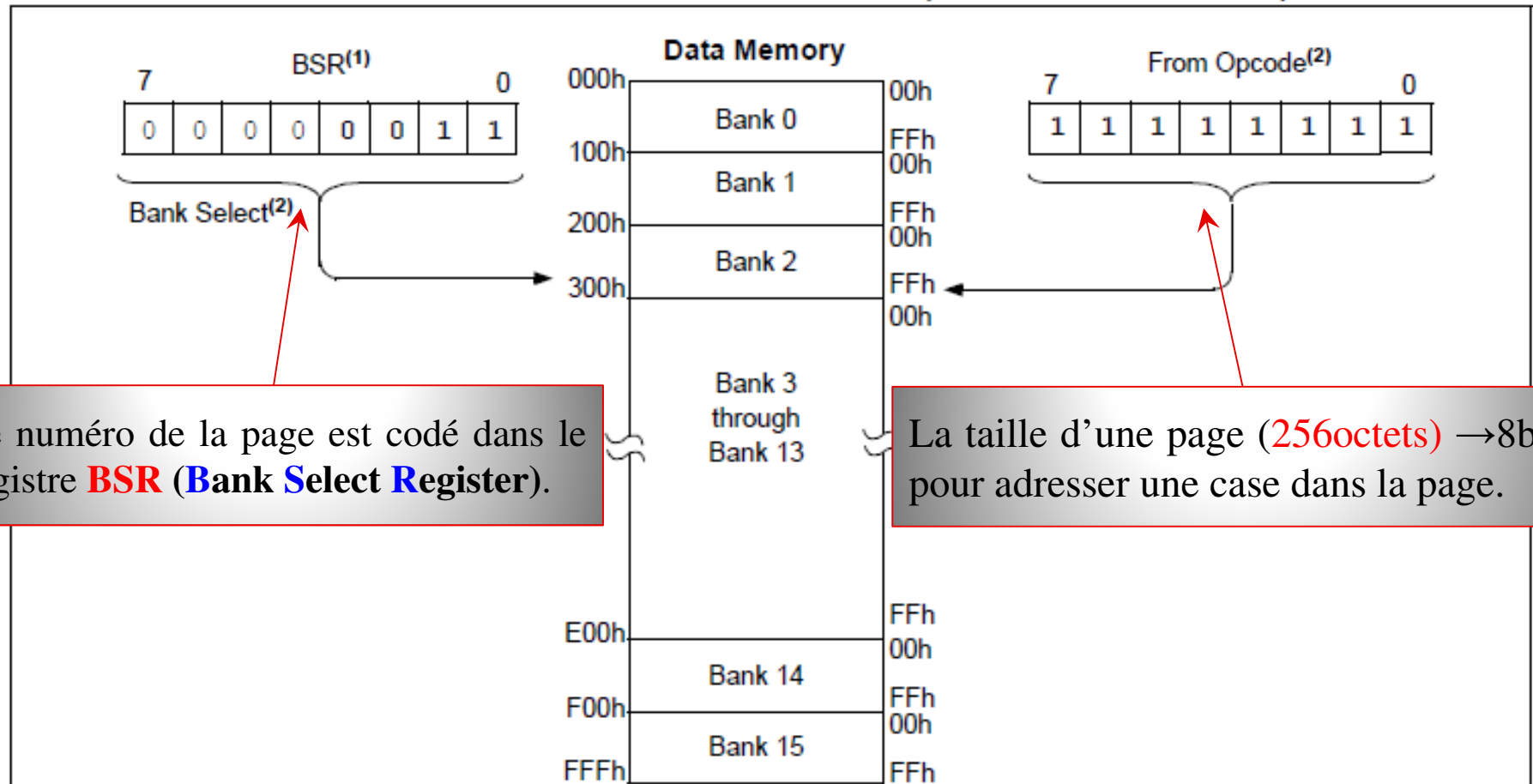
- La mémoire de données contient tout à la fois des registres de l'unité centrale, ceux de ressources internes ainsi que des registres à usage générale qui constituent en fait la véritable mémoire de données.
- Elle utilise un mécanisme de pagination. Elle est divisée en 15 pages (Microchip les appelle *bank* d'une taille fixe de **256 octets**).
- La mémoire vive ou mémoire de travail se trouve située dans les pages les plus basses c.à.d à partir de l'adresse 000 tandis que les registres de l'unité centrale et des ressources internes se trouvent tous groupés dans la page la plus haute (Page 15: de l'@F60 à FFF).



Organisation de la mémoire de données et modes d'adressage

datasheet, page : 67

FIGURE 5-6: USE OF THE BANK SELECT REGISTER (DIRECT ADDRESSING)

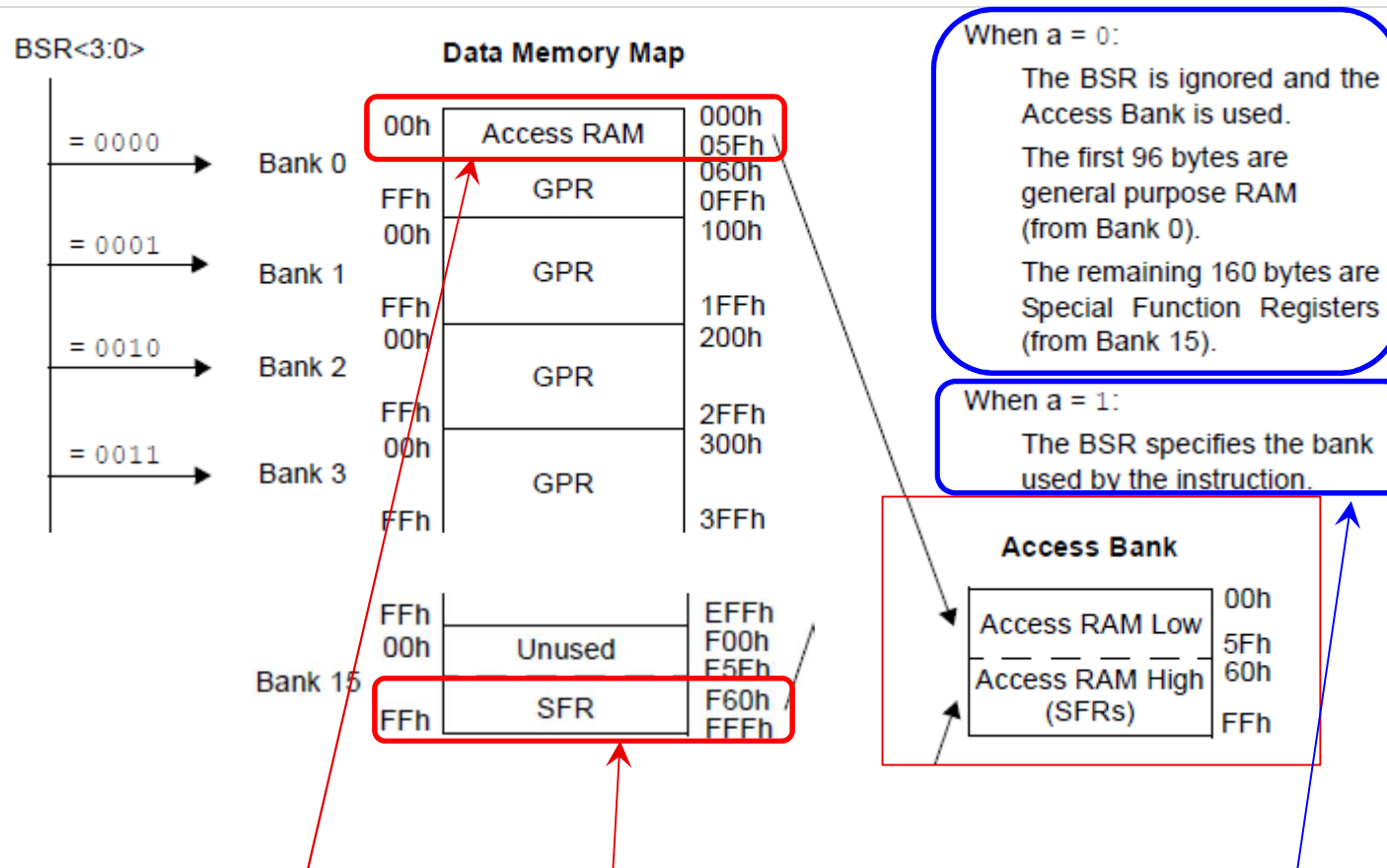


Le numéro de la page est codé dans le registre **BSR** (Bank Select Register).

La taille d'une page (**256octets**) → 8bits pour adresser une case dans la page.

- Note 1: The Access RAM bit of the instruction can be used to force an override of the selected bank (BSR<3:0>) to the registers of the Access Bank.
- Note 2: The `MOVEF` instruction embeds the entire 12-bit address in the instruction.

Organisation de la mémoire de données et modes d'adressage



Access Bank (Page d'accès): Toujours accessible $\forall$ le contenu de BSR par le suffixe «a» dans l'instruction:

➤ **a=0** → **Access Bank**

➤ **a=1** → **Banked**

Organisation de la mémoire de données et modes d'adressage

Les modes d'adressage

Adressage literal (ou immédiat)

La donnée manipulée par l'instruction est codée avec l'instruction elle-même.

Exemple :

MOVLW *k*

Place la valeur (ou *literal*) *k*, qui est une donnée codée sur 8bits, dans le registre de travail *W*.

Adressage direct

Consiste à coder l'adresse source ou destination utilisée par l'instruction directement dans le code de l'instruction elle-même.

Exemple :

MOVF *variable*, *W*

Charge le registre de travail *W* avec *variable*.

Jeu d'instructions du PIC18F4550

Architecture RISC

- Le PIC18F4550 est constitué autour d'une architecture appelée **RISC**.



Reduced Instruction Set Computer

→ Circuit à jeu d'instructions réduit → 75 instructions seulement!

Chaque **instruction est codée sur un mot de 16 bits** qui contient le **code opération** (OpCode : OC) ainsi que l'**opérande**.

Each single-word instruction is a 16-bit word divided into an opcode, which specifies the instruction type and one or more operands, which further specify the operation of the instruction.

datasheet, page 313

Jeu d'instructions du PIC18F4550

Les instructions

TABLE 26-2: PIC18FXXX

Mnemonic, Operands	Description		Instruction Word				Status Affected	
			MSb			LSb		
BYTE-ORIENTED OPERATIONS								
ADDWF	f, d, a	Add WREG and f	1	0010	01da	ffff	ffff	C, DC, Z, OV, N
ADDWFC	f, d, a	Add WREG and Carry bit to f	1	0010	00da	ffff	ffff	C, DC, Z, OV, N
ANDWF	f, d, a	AND WREG with f	1	0001	01da	ffff	ffff	Z, N
CLRF	f, a	Clear f	1	0110	101a	ffff	ffff	Z
COMF	f, d, a	Complement f	1	0001	11da	ffff	ffff	Z, N
CPFSEQ	f, a	Compare f with				ffff	ffff	None
CPFSGT	f, a	Compare f with				ffff	ffff	None
CPFSLT	f, a	Compare f with WREG, Skip <	1 (2 or 3)	0110	000a	ffff	ffff	None
DECF	f, d, a	Decrement f	1	0000	01da	ffff	ffff	C, DC, Z, OV, N
DECFSZ	f, d, a	Decrement f, Skip if 0	1 (2 or 3)	0010	11da	ffff	ffff	None
DCFSNZ	f, d, a	Decrement f, Skip if Not 0	1 (2 or 3)	0100	11da	ffff	ffff	None
INCF	f, d, a	Increment f						C, Z, OV, N
INCFSZ	f, d, a	Increment f, Skip if 0						
INFSNZ	f, d, a	Increment f, Skip if Not 0	1 (2 or 3)	0100	10da	ffff	ffff	None
IORWF	f, d, a	Inclusive OR WREG with f	1	0001	00da	ffff	ffff	Z, N
MOVF	f, d, a	Move f	1	0101	00da	ffff	ffff	Z, N
MOVFF	f _s , f _d	Move f _s (source) to 1st word f _d (destination) 2nd word	2	1100	ffff	ffff	ffff	None
MOVWF	f, a	Move WREG to f	1	0110	111a	ffff	ffff	None

Les instructions du PIC18F4550

Les opérandes

Les codes d'opération (Opcode)

datasheet, pages: 316-318

Les instructions du
PIC18F4550

Les opérandes

Les codes d'opération (Opcode)

datasheet, pages: 316-318

Jeu d'instructions du PIC18F4550

Conventions pour les opérandes

MOVLW **(k)**

ADDWF **(f,d,a)**

datasheet, page : 314

- « **k** » exprime une donnée (valeur),
- « **(var)** » Signifie la « valeur contenue à l'adresse **var** »
- « **d** » (**destination**) exprime l'endroit où le résultat de l'opération sera placé. Ce résultat pourra être soit « **f ou 1** », soit « **w ou 0** ».
- « **f** » indique que la destination est un « **file register** », signifie simplement que la destination est le registre précisé dans l'instruction.
- « **w** » précise que le résultat de l'opération se trouve dans le registre **W** (= WREG). Dans ce cas, l'opération ne modifie pas le registre éventuellement précisé dans l'instruction.
- « **a** » représente le type d'adressage direct utilisé, soit en **access bank** «**a=0**» , soit en **banked** « **a=1** » .

Jeu d'instructions du PIC18F4550

Les instructions

Description des opérations
réalisées par chaque
instruction

Nombre de cycles
d'horloge pour
chaque instruction

Code de
l'instruction

Etat du registre STATUS
après l'exécution de
l'instruction

TABLE 26-2. PIC18FXXX INSTRUCTION SET

Mnemonic, Operands	Description	Cycles	16-Bit Instruction Word				Status Affected
			MSb		LSb		
BYTE-ORIENTED OPERATIONS							
ADDWF f, d, a	Add WREG and f	1	0010	01da	ffff	ffff	C, DC, Z, OV, N
ADDWFC f, d, a	Add WREG and Carry bit to f	1	0010	00da	ffff	ffff	C, DC, Z, OV, N
ANDWF f, d, a	AND WREG with f	1	0001	01da	ffff	ffff	Z, N
CLRF f, a	Clear f	1	0110	101a	ffff	ffff	Z
COMF f, d, a	Complement f	1	0001	11da	ffff	ffff	Z, N
CPFSEQ f, a	Compare f with WREG, Skip =	1 (2 or 3)	0110	001a	ffff	ffff	None
CPFSGT f, a	Compare f with WREG, Skip >	1 (2 or 3)	0110	010a	ffff	ffff	None
CPFSLT f, a	Compare f with WREG, Skip <	1 (2 or 3)	0110	000a	ffff	ffff	None
DECF f, d, a	Decrement f	1	0000	01da	ffff	ffff	C, DC, Z, OV, N
DECFSZ f, d, a	Decrement f, Skip if 0	1 (2 or 3)	0010	11da	ffff	ffff	None
DCFSNZ f, d, a	Decrement f, Skip if Not 0	1 (2 or 3)	0100	11da	ffff	ffff	None
INCF f, d, a	Increment f	1	0010	10da	ffff	ffff	C, DC, Z, OV, N
INCFSZ f, d, a	Increment f, Skip if 0	1 (2 or 3)	0011	11da	ffff	ffff	None
INFSNZ f, d, a	Increment f, Skip if Not 0	1 (2 or 3)	0100	10da	ffff	ffff	None
IORWF f, d, a	Inclusive OR WREG with f	1	0001	00da	ffff	ffff	Z, N
MOVF f, d, a	Move f	1	0101	00da	ffff	ffff	Z, N
MOVFF f _s , f _d	Move f _s (source) to 1st word f _d (destination) 2nd word	2	1100	ffff	ffff	ffff	None
			1111	ffff	ffff	ffff	
MOVWF f, a	Move WREG to f	1	0110	111a	ffff	ffff	None

Jeu d'instructions du PIC18F4550

Les instructions

Cycles

Pour un quartz de 4MHz, l'horloge système (instruction) est de $f_{osc}/4 = 1000000$ cycles/seconde
→ une puissance de calcul de 1MIPS (1 Million Instructions Par Seconde).

Avec un quartz de 20MHz, on obtient une vitesse de traitement de 5 MIPS.

Les Indicateurs d'état

Le rôle du registre **STATUS** est de donner diverses informations à l'utilisateur sur l'état de fonctionnement ou sur le résultat d'une opération.

REGISTER 5-2: STATUS REGISTER

datasheet, page : 73

U-0	U-0	U-0	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
—	—	—	N	OV	Z	DC ⁽¹⁾	C ⁽²⁾
bit 7			bit 0				

- **Z** (Zero) : passe à 1 quand le résultat d'une instruction est nul
- **C** (Carry) : passe à 1 quand l'opération a généré une retenue
- **DC** (Quartet Carry): passe à 1 quand le 4ème bit génère une retenue
- **N** : si négatif
- **OV** : s'il y a eu un débordement dans une opération en complément à 2

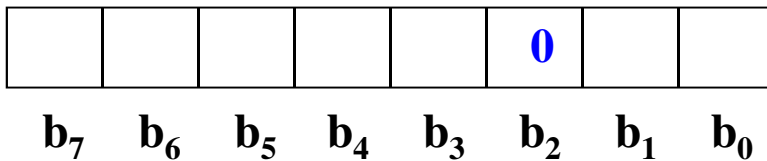
Ces indicateurs permettent de savoir comment une instruction s'est terminée. Toutes les instructions n'agissent pas sur les indicateurs.

Jeu d'instructions du PIC18F4550

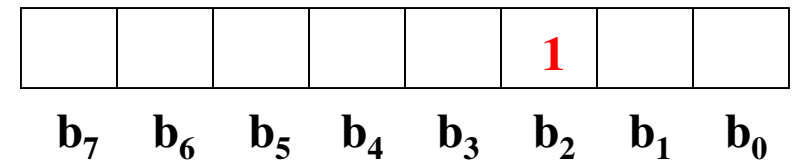
Les Indicateurs d'état

Le bit b_2 : Z (Zéro) : Lorsqu'une opération arithmétique ou logique est réalisée, le bit **Z** est mis à **1** si le résultat est nul et à **0** dans le cas contraire.

Exp1: L'unité centrale effectue l'opération $7-6=1$.

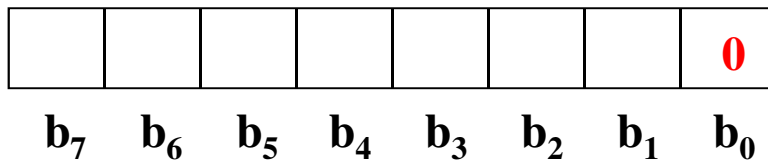


Exp2: L'unité centrale effectue l'opération $7-7=0$.

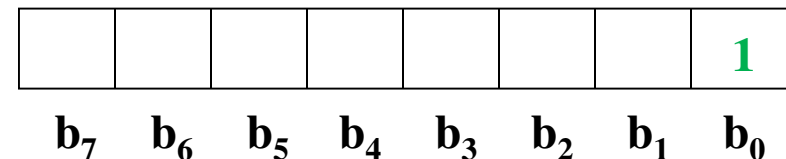


Le bit b_0 : C (Carry = retenue) : Ce bit est positionné à **1** si une addition ou une soustraction génère une retenue depuis le bit de poids fort.

Exp1: L'unité centrale effectue l'opération sur 8 bits: $FE+01=FF$



Exp2: L'unité centrale effectue l'opération hexadécimale: $FF+1=00$ (et une retenue)



Jeu d'instructions du PIC18F4550

Types des instructions

Les instructions « orientées Registre »

Les instructions « orientées bits »

Les instructions opérant sur une constante

Les instructions de saut et appel de procédures

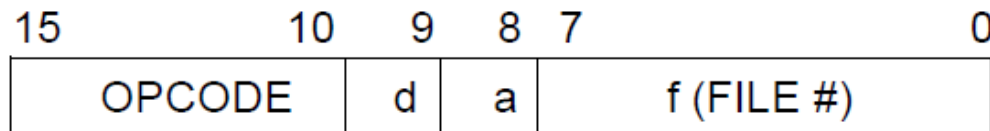
Les instructions « orientées Registres »

Format de l'instruction

Ce sont des instructions qui manipulent un **octet** se trouvant dans la RAM. Ça peut être un **registre de configuration SFR** ou une **case mémoire** quelconque (Registre **GPR**).

Byte-oriented file register operations

datasheet, page : 315



d = 0 for result destination to be WREG register **Destination Résultat dans WREG**

d = 1 for result destination to be file register (f) **Destination Résultat dans un registre (f)**

a = 0 to force Access Bank **Adressage « access Bank »**

a = 1 for BSR to select bank **Adressage « Banked »**

f = 8-bit file register address **Registre sur 8 bits**

Exp1:

ADDWF 8Eh,1 ou **ADDWF 8Eh,f**

Syntaxe:

ADDWF f, d, a | Add WREG and f

Additionner le contenu de W avec le contenu de la case mémoire d'adresse 8Eh et placer le résultat dans la case mémoire 8Eh.

Exp2:

XORWF AFh,0 ou **XORWF AFh,w**

Syntaxe:

XORWF f, d, a | Exclusive OR WREG with f

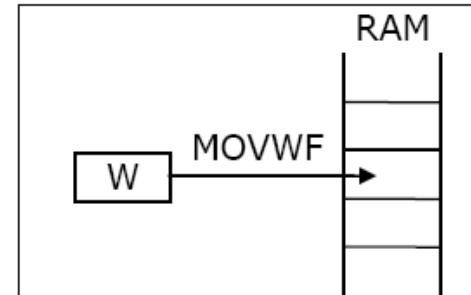
Faire un ou exclusif entre W et le contenu de la case mémoire d'adresse AFh et placer le résultat dans l'accumulateur W

Les instructions « orientées Registres »

L'instruction **MOVWF**

Syntaxe: `MOVWF f, a` | Move WREG to f

MOVWF permet de copier l'accumulateur **W** dans un registre (**SFR** ou **GPR**).



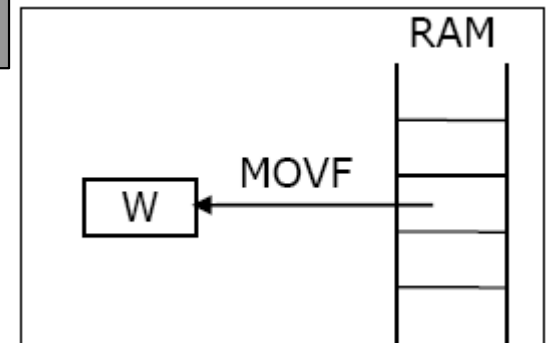
Exemple 1: `MOVWF STATUS` Copier le contenu de **W** dans le registre **STATUS**

Exemple 2: `MOVWF 55h` Copier le contenu de **W** dans la case mémoire d'adresse **55h**

L'instruction **MOVF**

Syntaxe: `MOVF f, d, a` | Move f

MOVF permet de copier le contenu d'un registre (**SFR** ou **GPR**) dans l'accumulateur **W**, le paramètre **d** doit être = 0



Exemple 1: `MOVF STATUS, 0` Copier le contenu du registre **STATUS** dans l'accumulateur **W**

Exemple 2: `MOVF 55h` Copier le contenu de la case mémoire d'adresse **55h** dans l'accumulateur **W**

Les instructions « orientées Registres »

L'instruction MOVFF

Syntaxe:

MOVFF	f_s, f_d	Move f_s (source) to 1st word
		f_d (destination) 2nd word

Move File to File (copier le registre dans un autre registre).

Copie un registre dans un autre registre, à travers toute la mémoire RAM, et sans restriction de banque.

Exemple : Soit une « variable » située dans la banque 3

MOVF variable, w Charge variable dans **W**, BSR doit pointer sur la banque 3

MOVFF variable, WREG Charge variable dans **W** indépendamment de BSR

MOVWF variable Sauver **W** dans variable, BSR doit pointer sur la banque 3

MOVFF WREG, variable Sauver **W** dans variable indépendamment de BSR

Les instructions « orientées Registres »

L'instruction ADDWF

Syntaxe: `ADDWF f, d, a | Add WREG and f`

Effectue une addition **sur 8 bits** entre le registre désigné (**f**) et le registre **W**. Le résultat sera placé **soit dans W, soit dans f**, selon le paramètre précisé comme destination.

Notez que cette syntaxe précise que le paramètre «d» est facultatif, ainsi que le paramètre «a». Par contre, vous voyez que pour pouvoir préciser le paramètre «a», vous êtes contraint également de préciser le paramètre de destination.

ADDWF	ADD W to f
Syntax:	<code>ADDWF f {,d {,a}}</code>
Operands:	$0 \leq f \leq 255$ $d \in [0,1]$ $a \in [0,1]$
Operation:	$(W) + (f) \rightarrow \text{dest}$
Status Affected:	N, OV, C, DC, Z

datasheet, page : 319

Exemple:

`ADDWF variable ;` OU `ADDWF variable,f ;` $\Rightarrow$ Résultat dans f.

`ADDWF variable,f,a ;` $\Rightarrow$ Résultat dans f, f est en *access bank*.

~~`ADDWF variable,a`~~ $\Rightarrow$ **Interdit**, car a est précisé et pas d.

Plan du Cours

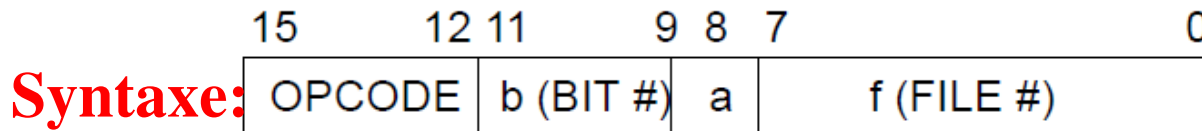
- Jeu d'instructions du PIC18F4550
- Les instructions « orientées Registres»
- **Les instructions « orientées Bits»**
- Les instructions de saut et appel de procédures
- Les instructions opérant sur une constante

Les instructions « orientées bits »

Format de l'instruction

Pour les instructions agissant sur un bit, le paramètre **F** indique le registre qui contient le bit à modifier et le paramètre **b** indique le numéro du bit à modifier; on compte à partir de zéro en commençant à droite :

Bit-oriented file register operations



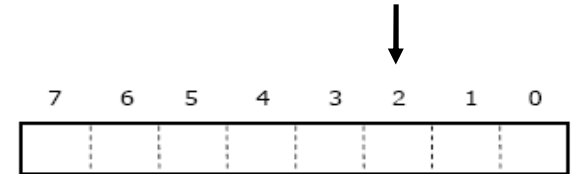
- b = 3-bit position of bit in file register (f) **Position du bit de 0 à 7 dans le registre f**
- a = 0 to force Access Bank **Adressage « access Bank »**
- a = 1 for BSR to select bank **Adressage « Banked »**
- f = 8-bit file register address **Fichier sur 8 bits**

datasheet, page : 315

Les instructions « orientées bits »

Exemple 1: **BSF STATUS,2**

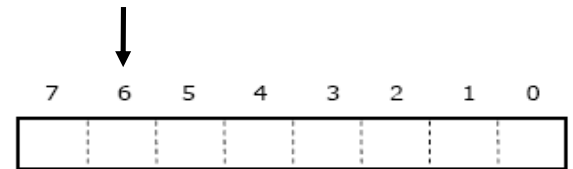
Syntaxe: BSF f, b, a | Bit Set f



Placer à 1 le bit 2 (3ème bit à partir de la droite) du registre STATUS.

Exemple 2: **BCF 45h,6**

Syntaxe: BCF f, b, a | Bit Clear f



Placer à 0 le bit 6 (7ème bit à partir de la droite) du registre de la case mémoire d'adresse 45h.

Les instructions « orientées bits »

L'instruction **BTFSC**

Syntaxe:

BTFSC f, b, a | Bit Test f, Skip if Clear

Teste le bit **b** du registre **F** et saute l'instruction suivante si le bit testé est **nul**.

L'instruction **BTFSS**

Syntaxe:

BTFSS f, b, a | Bit Test f, Skip if Set

Teste le bit **b** du registre **F** et saute l'instruction suivante si le bit testé est égal à **1**.

Exemple:



```
sublw 100 ;      100 - W → W
btfs STATUS,Z ;  tester le bit Z du
                  registre STATUS et
                  sauter une ligne si
                  Z=1

clrf 70h ;        après btfs, le
                  programme
                  continue ici si Z=0

cmpf 70h,f ;      après btfs, le
                  programme
                  continue ici si Z=1

suite du programme

suite du programme

...
```

Plan du Cours

- Jeu d'instructions du PIC18F4550
- Les instructions « orientées Registres»
- Les instructions « orientées Bits»
- **Les instructions de saut et appel de procédures**
- Les instructions opérant sur une constante

Les instructions de saut et appel de procédures

L'instruction **goto**

L'instruction **goto** Permet de transférer l'exécution à une autre position du programme repérée par une étiquette (**label**).

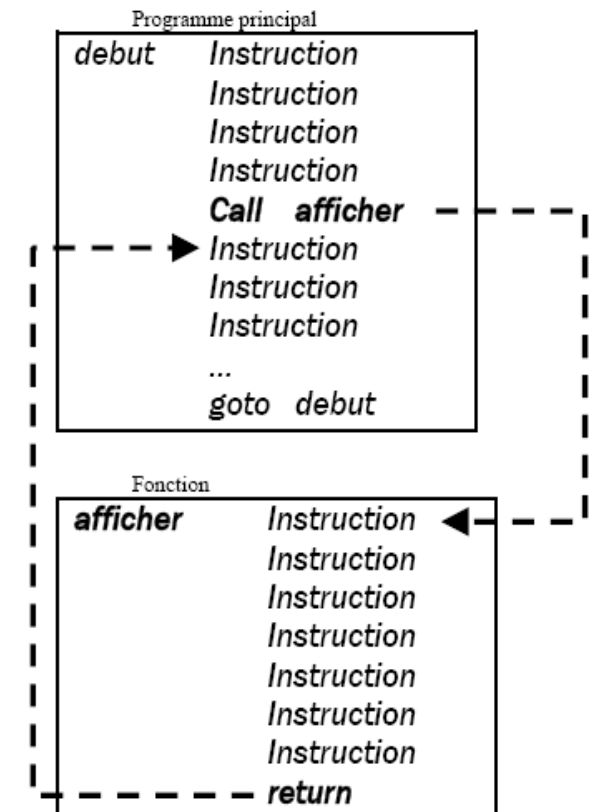
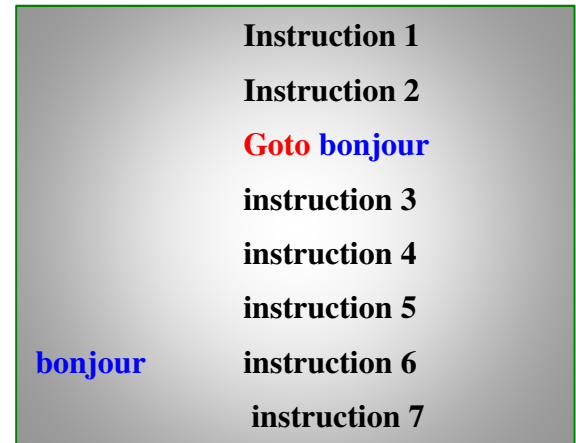
L'instruction **call**

L'instruction **call** permet d'appeler une fonction. Une fonction est un sous programme écrit à la suite du programme principal. Sa première ligne doit comporter une étiquette et elle doit se terminer par **return**.

Différence entre **CALL** et **GOTO**:

Quand le processeur rencontre l'instruction **call**, il sauvegarde l'adresse de la ligne suivante avant d'aller exécuter les instructions constituant la fonction.

Comme ça, quand il rencontre l'instruction **return**, il sait où il doit retourner pour continuer l'exécution du programme principal.



Plan du Cours

- Jeu d'instructions du PIC18F4550
- Les instructions « orientées Registres»
- Les instructions « orientées Bits»
- Les instructions de saut et appel de procédures
- **Les instructions opérant sur une constante**

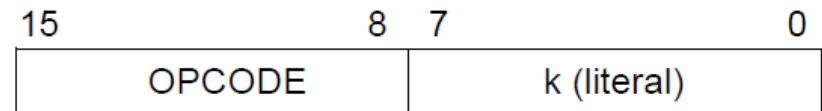
Les instructions « orientées bits »

Format de l'instruction

Ce sont les instructions entre l'accumulateur **W** et une constante **k**.

Exemple: `MOVLW 7Fh`

Literal operations



k = 8-bit immediate value

L'instruction **SUBLW**

Syntaxe:

SUBLW	k	Subtract WREG from Literal
-------	---	----------------------------

Soustraire le contenu de **W** du littéral **k** et placer le résultat dans **W**.

L'instruction **MULLW**

Syntaxe:

MULLW	k	Multiply Literal with WREG
-------	---	----------------------------

Multiplier le contenu de **W** par le littéral **k** et placer le résultat dans **W**.